1) Object-Oriented Programming (OOP) :

- Object-Oriented Programming (OOP) is a programming paradigm that organizes code into reusable and modular structures called **objects**, created from **classes**. It simplifies software development by mimicking real-world entities.
- Object-Oriented Programming (OOP) combines data and actions into meaningful structures using **classes** and **objects**. By following its four pillars—**Inheritance**, **Polymorphism**, **Encapsulation**, and **Abstraction**—OOP helps developers create programs that are modular, reusable, and easy to maintain. These concepts ensure better code organization, security, and adaptability, making OOP a powerful approach to software development.

2) Four Pillars of OOP

i. Inheritance

- **Definition**: Inheritance allows one class (child) to inherit the properties and methods of another class (parent). This creates a hierarchy and promotes code reuse.
- **Types of Inheritance**:
    - **Single**: A child class inherits from one parent class (e.g., Car inherits from Vehicle).
    - **Multilevel**: A chain of inheritance where a child inherits from a parent, and another child inherits from that child (e.g., Vehicle → Car → ElectricCar).
    - **Hierarchical**: Multiple child classes inherit from the same parent class (e.g., both Car and Bike inherit from Vehicle).

- **Multiple**: A child class inherits from multiple parent classes (e.g., a FlyingCar inherits from both Car and Airplane).
- **Hybrid**: A mix of two or more inheritance types.

- **Why It's Needed**:
  - Reduces code duplication by reusing common features.
  - Organizes code logically by creating relationships between classes.
  - Provides flexibility to extend or override existing functionality.

---

ii. Polymorphism

- **Definition**: Polymorphism allows methods or actions to perform differently based on the object calling them. It provides flexibility by letting the same method behave differently.
- **Types of Polymorphism**:
  - **Method Overloading**: A method behaves differently depending on the number or type of parameters. Python achieves this through default arguments, *args, or **kwargs (e.g., a calculate method might add two numbers or compute an average for three numbers).
  - **Method Overriding**: A child class redefines a method from the parent class to customize its behavior (e.g., a withdraw method in a SavingsAccount might handle fees, unlike the BankAccount class).
- **Why It's Needed**:
  - Simplifies adding new features by reusing method names with different implementations.

Credence IT Professional Training Institute

- Increases flexibility, as objects can define their own behavior for shared methods.

---

iii. Encapsulation

- **Definition**: Encapsulation bundles data (attributes) and methods (actions) into a single unit (class). It restricts access to sensitive parts of an object using **public**, **protected**, and **private** access modifiers.
- **Access Modifiers**:
  - **Public**: Accessible from anywhere. Used for attributes or methods meant for general use (e.g., account_holder_name).
  - **Protected**: Accessible within the class and its subclasses. Denoted with a single underscore (_attribute) (e.g., _account_number).
  - **Private**: Strictly accessible within the class. Denoted with double underscores (__attribute) (e.g., __balance).
- **Why It's Needed**:
  - Protects sensitive data from unauthorized changes.
  - Ensures controlled access to attributes using **getter** and **setter** methods.
  - Promotes clean and secure code.

Credence IT Professional Training Institute

---

iv. Abstraction

- **Definition**: Abstraction focuses on showing only the essential details of an object or system while hiding its complexity. It tells "what to do" without revealing "how to do it."

- **Implementation**: Achieved through abstract classes or interfaces, which define methods that subclasses must implement.
- **Why It's Needed**:
  - Reduces complexity by hiding unnecessary details from the user.
  - Simplifies interaction with the system by focusing on the core functionalities.
  - Encourages modular design by separating the interface from the implementation.

---

3) Core Concepts and Definitions

1. **Class**: A blueprint for creating objects, defining common properties (attributes) and actions (methods).
2. **Object**: An instance of a class that contains actual data and behaviors based on the class definition.
3. **Constructor**: A special method (__init__) that initializes an object's attributes when it is created.
4. **Instance Variable**: A variable unique to an object, storing its specific data.
5. **Instance Method**: A method that works on an instance of the class and can modify its attributes.
6. **Class Attribute**: A variable shared by all objects of the class, such as a bank's name shared by all bank accounts.
7. **Class Method**: A method that operates on the class itself, modifying or accessing class-level data.

8. **Static Method**: A utility method that doesn't depend on class or instance data. It performs general tasks.
9. **Public Attribute/Method**: Fully accessible from anywhere.
10. **Protected Attribute/Method**: Intended for internal or subclass use; denoted with _.
11. **Private Attribute/Method**: Accessible only within the class; denoted with __.

---

4) Additional Concepts with Examples

1. **Variable**: A container for storing data values.
    - **Example**: x = 10 stores the value 10 in the variable x.
2. **Instance Variable**: A variable specific to an object, storing data unique to it.
    - **Example**: In a BankAccount object, the balance is an instance variable unique to each account.
3. **Attribute**: A variable defined within a class or object.
    - **Example**: A Car class might have an attribute color to store the car's color.
4. **Parameter**: A placeholder in a method definition that receives input.
    - **Example**: In def deposit(amount):, amount is a parameter.
5. **Arguments**: The actual values passed to a method when it is called.
    - **Example**: deposit(500) passes 500 as an argument.
6. **Keyword Arguments**: Arguments paired with parameter names for clarity.
    - **Example**: deposit(amount=500) explicitly names the parameter.

Credence IT Professional Training Institute

7. **Getter and Setter Methods**: Methods used to access or modify private attributes.
    - **Example**: A get_balance method retrieves a bank account's balance, while set_balance updates it.

---

5) Why Use OOP Concepts?

1. **Inheritance**:
    - Promotes code reuse and logical organization.
    - Reduces redundancy by sharing features across related classes.
2. **Polymorphism**:
    - Simplifies code reuse with shared method names that adapt to different objects.
    - Increases flexibility and scalability.
3. **Encapsulation**:
    - Secures sensitive data by restricting access.
    - Provides controlled access using getter and setter methods.
4. **Abstraction**:
    - Hides complex implementation details from the user.
    - Focuses on essential features, reducing complexity and improving usability.

Credence IT Professional Training Institute